

Apartado 1: Carga el dataset y prepara los datos

- Abre [el notebook CNN on MNIST](#) y revísalo entero. En principio, es un modelo demasiado sencillo, así que ahora te animamos a qué crees tu propio modelo.

Abro el notebook.

The screenshot shows a Google Colab notebook titled "CNN on MNIST.ipynb". The notebook content includes a title "Convolutional Neural Network on the MNIST Dataset", an introduction to the MNIST dataset, a sample image of the digit '8', and a mission statement. It then lists necessary libraries to import: tensorflow, keras, numpy, and pickle. The code block shows the following imports:

```
[ ] import tensorflow as tf
from tensorflow.keras.datasets import mnist #this library contains a lot of ML datasets including the MNIST one
from tensorflow.python.keras.models import Sequential
from tensorflow.python.keras.layers import Dense, Conv2D, Dropout, Flatten, MaxPooling2D
import pickle #pickle is a library that helps us save a lot of different types of data - anything ranging from Pandas dataframes to TensorFlow models
```

Below the code, there is a section titled "Formatting Data" explaining the importance of data formatting for deep learning models.

- Inicia un nuevo notebook, y carga el dataset desde Keras.

The screenshot shows a new Google Colab notebook titled "dieguez_alvarez_alberto_PIA_07_Tarea.ipynb". The notebook content includes a title "Tarea para PIA07" and a description "Convolutional Neural Network on the MNIST Dataset." It then lists the tasks: "Inicia un nuevo notebook, y carga el dataset desde Keras." The code block shows the following imports:

```
[ ] import numpy as np
from tensorflow.keras import keras
from keras.layers import layers
```

Below the code, there is a section titled "Formatting Data" explaining the importance of data formatting for deep learning models.

- Prepara los datos para poder ser ingeridos por una capa convolucional.

```

import numpy as np
from tensorflow import keras
from tensorflow.keras.datasets import mnist
from keras import layers

# Prepara los datos para poder ser ingeridos por una capa convolucional.

input_shape = (28, 28, 1)
(train_data, train_labels), (test_data, test_labels) = mnist.load_data()

train_data = train_data.reshape(train_data.shape[0], 28, 28, 1)
test_data = test_data.reshape(test_data.shape[0], 28, 28, 1)

train_data = train_data.astype('float32')
test_data = test_data.astype('float32')

train_data /= 255
test_data /= 255

print(train_data.shape[0], 'train samples')
print(test_data.shape[0], 'test samples')

```

Download data from <https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist-one>
 11496614/11496614 — 1s 8us/step
 60000 train samples
 10000 test samples

Apartado 2: Crea el modelo convolucional.

- Crea un modelo con una o varias capas Conv2D.
- Utiliza también capas max-pooling o strides que mantengan las dimensiones de la red bajo control.
- Recuerda añadir una capa neuronal interna y una capa neuronal de salida, con tantas neuronas como clases de salida necesitamos.

```

El código generado puede estar sujeto a licencia [Amikume7734/Crop_Recommendation_And_Disease_Detection]

model = Sequential()
model.add(Conv2D(28, kernel_size=(3,3), input_shape = input_shape)) # Input layer
model.add(MaxPooling2D(pool_size=(2, 2))) # downsizing images
model.add(Flatten()) # Images are 3d so have to Flatten them into a (1 x 784) vector
model.add(Dense(128, activation=tf.nn.relu, use_bias=True)) # adding a Dense layer of 128 neurons with relu
model.add(Dropout(0.5)) # Implementing dropout regularization with p = 0.5
model.add(Dense(78, activation=tf.nn.relu, use_bias=True))
model.add(Dropout(0.5))
model.add(Dense(10, activation=tf.nn.softmax)) # adding an output layer (with 10 possible outputs for the 10 digits we need to predict)
print(model.summary())

```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
conv2d_1 (Conv2D)	(None, 26, 26, 28)	288
max_pooling2d_1 (MaxPooling2D)	(None, 13, 13, 28)	0
flatten_1 (Flatten)	(None, 4732)	0
dense_3 (Dense)	(None, 128)	605824
dropout_2 (Dropout)	(None, 128)	0
dense_4 (Dense)	(None, 78)	9638
dropout_3 (Dropout)	(None, 78)	0
dense_5 (Dense)	(None, 10)	710
Total params:	615,844	
Trainable params:	615,844	
Non-trainable params:	0	

Apartado 3: Entrena y evalúa el modelo.

- Configura los parámetros del entrenamiento.
- Entrena el modelo.
- Evalúa el modelo con los datos de test y obtén la precisión real.
- ¿Es peor, igual o mejor que la que conseguimos en la unidad 5?.

Esta parte me ha dado muchos problemas. Al principio me daba error porque no era array, que no to_category... y más errores, al final encontré algo que funciona.

The screenshot shows a Google Colab notebook titled "dieguez_alvarez_alberto_PIA_07_Tarea.ipynb". The code defines a Keras model with the following layers:

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 28)	280
max_pooling2d (MaxPooling2D)	(None, 13, 13, 28)	0
flatten (Flatten)	(None, 4732)	0
dense (Dense)	(None, 128)	605,824
dropout (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 70)	9,830
dropout_1 (Dropout)	(None, 70)	0
dense_2 (Dense)	(None, 10)	710

Total params: 615,844 (2.35 MB)
Trainable params: 615,844 (2.35 MB)
Non-trainable params: 0 (0.00 B)

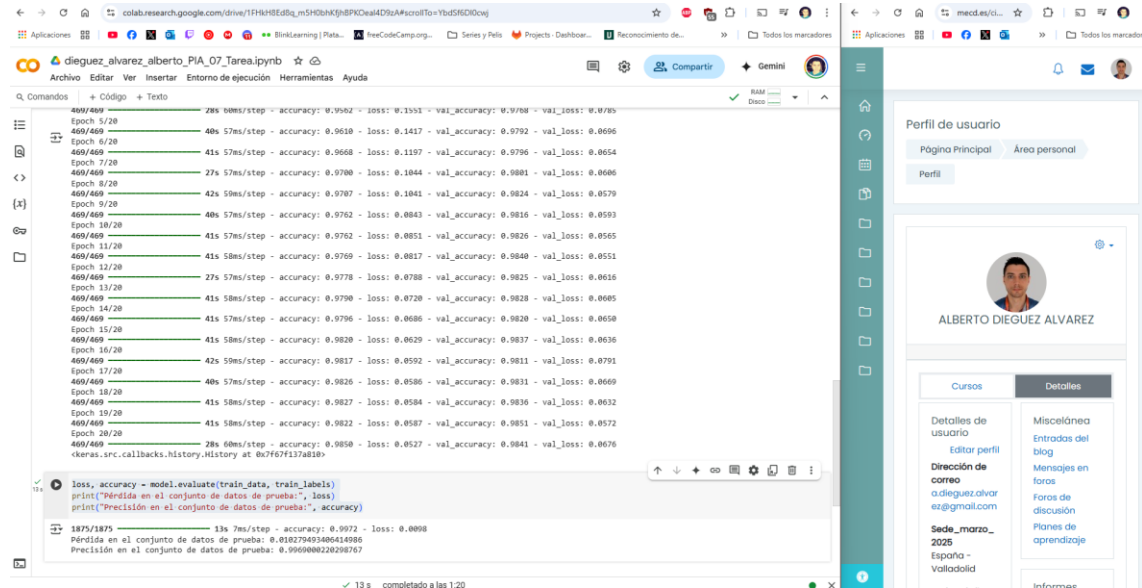
The training progress is shown as a bar chart with the following metrics:

Epoch	Step	Accuracy	Loss	Val Accuracy
469/469	30s 58ms/step	0.6885	0.9392	0.6885
469/469	27s 58ms/step	0.9234	0.2798	0.9234
469/469	28s 60ms/step	0.9453	0.2825	0.9453
41/469	21s 58ms/step	0.9543	0.1481	0.9543

An error message is displayed: "AttributeError: module 'tensorflow.python.distribute.input_lib' has no attribute 'DistributedDatasetInterface'". The suggested changes are:

```
import tensorflow as tf
import numpy as np
from tensorflow.keras.datasets import mnist
from tensorflow.python.keras.models import Sequential
from tensorflow.keras import layers
from tensorflow.keras.utils import to_categorical
```

Ahora comprobamos la pérdida y la precisión:



Muy buenos resultados.

Es mucho mejor que el de la tarea 5. Que los he visto y los de la tarea 5 son bastante malos. diferente a la precisión alcanzada en el entrenamiento?

```
loss, accuracy = model1.evaluate(X_test, y_test)
print("Pérdida en el conjunto de datos de prueba:", loss)
print("Precisión en el conjunto de datos de prueba:", accuracy)
```

```
313/313 3s 8ms/step - accuracy: 0.4915 - loss: 1.7106
Pérdida en el conjunto de datos de prueba: 1.7282911539077759
Precisión en el conjunto de datos de prueba: 0.4880000054836273
```

8. Explora de forma visual la precisión que se consigue, representando las primeras 25 imágenes del conjunto comparando la etiqueta real con la de la predicción. Aquí tienes una posible forma de hacerlo (recuerda p

Link al cuaderno: [Google Colab Notebook](#)